TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REGISTER

Class: **Duration time**: from 09/2026 To 03/2027

(*) Profession: Software Engineer **Specialty**: SE [x]

(*) Kinds of person make registers: Lecturer [x] Students

1. Register information for supervisor (if have)

No.	Full name	Phone	E-Mail	Title
Supervisor	Đặng Ngọc Minh Đức		ducdnm2@fe.edu.vn	Giảng viên
Co-supervisor	Bùi Văn Phúc			Giảng viên

2. Register information for students (if have)

No.	Full name	Student code	Phone	E-mail	Role in Group
1	Đỗ Đăng Khoa	SE192357		khoado7577@gmail.com	Leader
2	Trần Khải Hoàng	SE182767		HoangTKSE182767@fpt.edu.vn	Member
3	Lâm Phi Long	SE182769		LongLPSE182769@fpt.edu.vn	Member
4	Nguyễn Bá Tân	SE183860		TanNBSE183860@fpt.edu.vn	Member
5	Nguyễn Ngọc Long	SE193490		nguyenngoclong216@gmail.com	Member

3. Register content of Capstone Project

(*) 3.1. Capstone Project name:

3.1.1. English: ***TrekLink — An Integrated Off-Grid Communication and Smart Device Rental Management Platform for Trekking Agencies***

3.1.2. Vietnamese: ***TrekLink — Nền tảng tích hợp liên lạc ngoài vùng phủ sóng và quản lý cho thuê thiết bị thông minh cho doanh nghiệp trekking***

Abbreviation: TrekLink

(*) 3.2. Main proposal content (including result and product)

a) Context:

Trekking companies lead groups along mountain routes where mobile phone coverage is weak or missing for long stretches. Satellite messengers need both the device and a paid satellite

subscription [14], so they cost more to buy and run. The team sees room in this market for off-grid devices that a company rents instead of buying.

In Summer 2026 the project team developed TrekLink, an open-source LoRa mesh firmware for ESP32/ESP32-S3 devices built on Meshtastic [6]. It supports multi-hop mesh messaging, GPS tracking, fall detection and automatic SOS broadcasting. This firmware and the team's hardware are the technical foundation of the Capstone.

A gap remains between this field prototype and a platform that organizations can operate. The current needs are:

- **No rental or asset management:** a provider cannot track which organization holds which device, its condition, its charges or its maintenance history.
- **Unreliable event delivery:** field events produced while the internet is down are lost, because nothing stores and forwards them between the mesh and the cloud.
- **Uncoordinated incident response:** an SOS from a device starts no structured workflow, so acknowledgment, response and resolution leave no record.
- **Manual rental operations:** contracts, handover, return, invoices and payments run through disconnected manual records.
- **No field visibility:** an organization has no live view of its own devices, positions and alerts.

Scope Boundary: TrekLink owns the devices, the firmware, the Field Station software and the cloud platform, and rents the devices to corporate organizations as the sole provider. The platform manages devices and does not manage tours. Each organization keeps its trips, customers and rescue operations in its own systems. Targeted firmware enhancements made during the Capstone, such as a durable on-device event queue, are in scope. Rearchitecting the mesh stack is out of scope. TrekLink is a supplementary safety and coordination tool with no guaranteed rescue link: a device is heard only within LoRa range of another TrekLink node or a Field Station, and the Field Station reaches the cloud only while it has an internet connection. A device out of range keeps its events in its own queue until a node hears it, and a device that stops reporting raises a stale-device alert with its last known position. The Capstone covers the system's functions and does not verify hardware ratings such as radio range.

b) Proposed Solutions:

TrekLink extends the LoRa mesh firmware into a device rental platform built on asset management, through six core capabilities:

- **Organization and rental management:** Organizations register, and TrekLink verifies and approves them. They rent devices under contracts and rental plans whose terms and prices are configured per device and per plan. Handover and return take place in person at the TrekLink counter, and payment is made online or at the counter. Each organization can sort its rented devices into groups and label each device with its holder.
- **Asset management:** Each device carries a printed asset tag and its radio node number, and moves through a lifecycle from intake to disposal with controlled transitions, covering reservation, rental, return, inspection, maintenance and loss.

- **Offline-resilient Field Station:** The organization runs the Field Station on a laptop at a base point of its choice, with one rented node attached by USB. Each field event carries a unique event ID derived from identifiers the firmware already transmits. While the internet is unavailable, the Field Station stores events in a local SQLite queue and shows the local mesh on site. SOS events have the highest priority and are synchronized first on reconnection.
- **Idempotent event processing:** The backend uses the event ID as an idempotency key [9]. A repeated delivery of the same event creates no second incident and no second alert.
- **Incident management with tiered alerts:** Each SOS becomes one incident, routed to the organization that held the device at that moment. Alerts go to the organization's on-duty members and escalate through further tiers after configurable timeouts. An incident nobody acknowledges escalates to TrekLink Staff, who report it to the authorities.
- **Live telemetry:** Each organization sees only its own devices, positions, battery levels and alerts on a live map, and can read the same data through an API with an organization key.

Technology stack:

- **Embedded firmware:** ESP32/ESP32-S3 with the TrekLink firmware, with targeted enhancements
- **Field Station:** Node.js with TypeScript, with async I/O for LoRa serial and MQTT communication and a local SQLite queue
- **Backend:** NestJS with TypeScript, a modular architecture with REST and WebSocket support
- **Database:** PostgreSQL with the Prisma ORM, for ACID rental transactions and the incident audit trail
- **IoT messaging:** MQTT (Mosquitto), QoS-based delivery between the Field Station and the backend [4]
- **Real-time push:** Socket.io WebSocket, for live dashboard updates and the organization API stream
- **Frontend:** React and TypeScript, role-based views with Leaflet map integration
- **Payment:** an online payment gateway in its sandbox, and payment at the counter
- **Hosting and DevOps:** AWS as the target hosting of the system, Docker Compose for containerised deployment and GitHub Actions for CI/CD

c) Functional Requirements:

- **TrekLink Admin:**

• Approve, reject, suspend and reactivate organizations.

• Manage platform accounts, roles, permissions and system configurations.

• Manage hardware variants, firmware versions, rental plans and prices.

• Approve the damage and maintenance charges that need Admin approval, and retire devices.

• Monitor system health, Field Station connectivity and incidents, and view audit logs and reports.

- **TrekLink Staff:**

• Verify organizations and record their signed contracts.

• Register devices, run intake checks and record maintenance.

• Approve rental contracts and reserve devices for them.

• Provision devices for the renting organization and hand them over at the counter.

• Check devices in, inspect them, record damage and loss, and close contracts.

• Record counter payments.

• Handle escalated incidents and record reports to the authorities.

- **Organization Manager:**

• Register the organization and request rental contracts.

• Pay invoices online or at the counter, and view contracts, invoices and payments.

• Manage the organization’s members, on-duty roster and API keys.

• Manage the organization’s rented devices, their groups and holder labels, and report a fault or a loss.

- **Organization Operator:**

• Receive SOS alerts, acknowledge incidents and report the response status and outcome.

• View the organization’s devices, positions, battery levels and alerts on the live map.

• Run the Field Station at the base point and view the local mesh offline.

- **Organization API client:**

• Read the organization’s positions, telemetry and incident events over REST and WebSocket with an organization API key.

- **Asset Management:**

• Register each device with its asset tag, node number, hardware variant, firmware version and status.

• Manage the device lifecycle through its allowed transitions only.

• Handle a lost device as a loss case decided from the rental history.

• Record battery level, last-seen time, latest GPS position and telemetry status.

• Keep the custody, contract, incident, inspection and maintenance histories of each device.

• Prevent unavailable or due-for-service devices from being allocated.

- **Event Reliability & Offline Synchronization:**

• Each field event shall carry a unique event ID, the source device, the event type, a UTC timestamp and a priority tier.

• The Field Station shall persist unsent events in a local SQLite queue while cloud connectivity is unavailable.

• On reconnection, SOS events shall be delivered before lower-priority events such as GPS updates and routine telemetry.

• The Field Station shall flush queued events automatically and report its synchronization status and queue depth per priority tier.

• The backend shall process each incoming event idempotently, with the event ID as the idempotency key.

• Repeated delivery of the same SOS event shall create exactly one incident and one logical notification.

- **Live Telemetry & Incident Management:**

• Display each organization’s devices, positions, status and last-seen time on a real-time map, scoped to that organization on the server.

• Create an incident automatically from a valid SOS event and route it to the organization that held the device.

• Alert the on-duty members first and escalate through further tiers after configurable timeouts, ending with TrekLink Staff.

• Support acknowledgment, response, resolution, false alarm, escalation, authority report and closing.

• Record the actor, the role, a timestamp and an action note at each transition, in an append-only history.

- **Billing & Reporting:**

• Calculate rental charges per device and per plan, together with late, damage and loss charges.

• Generate invoices and track payment status.

• Accept online payments through a payment gateway in its sandbox, and record counter payments.

• Generate reports on contracts, fleet utilization, revenue, maintenance, incidents and response performance.

d) Non-Functional Requirements:

- **API Performance:** At least 95% of normal backend API requests shall complete within 300 ms at 50 concurrent users.
- **Synchronization Latency:** While the Field Station uplink is available, new field events shall reach the backend within 5 seconds under normal load.
- **Offline Recovery:** At least 99% of the events produced during a connectivity loss of 30 s to 30 min shall be delivered after recovery, over at least 20 trials per condition.
- **Testability:** Connectivity loss shall be reproducible in the lab by interrupting the Field Station's network interface programmatically.
- **Duplicate Prevention:** Ten deliveries of one SOS event shall create one incident, verified by an automated test.
- **Priority Ordering:** SOS events queued during an outage shall be delivered before GPS and routine telemetry events in at least 99% of trials.
- **Concurrency:** The backend shall process at least 20 simultaneous Field Station submissions without loss or duplication.
- **Offline Resilience:** Loss of cloud connectivity shall not interrupt local mesh communication or SOS propagation between devices.
- **Security:** JWT authentication, RBAC, TLS transport and bcrypt password hashing. An organization shall not be able to read another organization's devices, positions, incidents, contracts or invoices, and a cross-organization test per module shall verify this.
- **Data Consistency:** Replayed Field Station messages shall not create duplicate incidents or duplicate telemetry records.
- **Auditability:** Incident, device and contract transitions shall be traceable to an actor, a role and a UTC timestamp.
- **Scalability:** The system shall support at least 10 organizations and 50 rented devices without architectural change.
- **Deployability:** Server components shall run in containers with Docker Compose and be tested and deployed through GitHub Actions.

e) Theory & Practical:

Theory: The project applies software engineering and distributed systems principles to integrate an intermittently connected LoRa mesh network with a cloud-based operational platform.

- **Finite State Machines:** Model the device, incident, organization and rental contract lifecycles with controlled, auditable transitions [8].
- **Distributed Systems and Eventual Consistency:** MQTT loosely couples the Field Station and the backend, and the backend converges to a consistent state once queued events arrive [7].

- **Priority-Aware Store-and-Forward:** Classify field events by safety criticality and flush the queue in that order, so SOS events are not delayed behind routine data [5].
- **Idempotent Event Processing:** Handlers produce the same outcome however many times an event is delivered, with the event ID as a natural idempotency key [9]. Reliability is handled above the transport layer.
- **IoT Messaging Protocols:** MQTT QoS 1 delivers each message at least once [4]. Backend idempotency turns this into effectively exactly-once processing.
- **Event-Driven Architecture:** An SOS starts a pipeline of ingestion, idempotency check, incident creation, notification and audit logging, with stages that can be tested independently.
- **Role-Based Access Control and Multi-Tenancy:** The API enforces permissions for TrekLink and organization roles, and the server isolates each organization's data.
- **Software Architecture:** A modular NestJS design separates event synchronization, organizations, devices, rental contracts, incidents, monitoring, billing and platform services.

Practical: Build on the team's ESP32/ESP32-S3 LoRa mesh firmware and hardware, and document the message schema and event ID structure before Field Station implementation begins.

- Develop the Field Station in Node.js and TypeScript: serial message parser, SQLite priority queue, MQTT publisher, reconnection flush and health reporting.
- Develop the NestJS backend modules for authentication and RBAC, organizations, devices, rental contracts, event synchronization, incidents, billing and reporting, with PostgreSQL migrations through Prisma.
- Develop the React web client with role-based views, a Leaflet live map, an incident dashboard, contract and asset management and reports.
- Deploy with Docker Compose to AWS and automate testing and deployment with GitHub Actions.
- Run the experiments for RQ1, RQ2 and RQ3 described in section 3.3.

f) Products (Expected Deliverables):

- **TrekLink Rental Platform:** web application for TrekLink Admin and Staff and for each organization's Manager and Operators.
- **Field Station:** LoRa-to-cloud gateway with a persistent SQLite priority queue, MQTT synchronization, reconnection flush, health reporting and a local offline view.
- **Event Reliability Layer:** event ID scheme, backend idempotency, synchronization audit log and duplicate-prevention tests.
- **Organization & Rental Contract Module:** organization onboarding, members, on-duty roster, API keys, rental plans, contracts and device groups.
- **Asset Management Module:** device lifecycle, intake, provisioning, inspection, maintenance and loss cases.
- **SOS Incident Management Module:** automatic incident creation, tiered alerts with escalation to TrekLink, response tracking and audit history.

- **Live Telemetry Module:** organization-scoped live map, telemetry, positions and the organization API.
- **Billing Module:** charges, invoices, sandbox online payments and counter payments.
- **Technical Documentation:** SRS with UML state machine diagrams, architecture, database schema, API reference, test plan, deployment guide and user guide.
- **Evaluation Report:** experimental results for RQ1, RQ2 and RQ3.
- **Source Code & Deployment Package:** repositories for the Field Station, backend and frontend, Docker Compose configuration, GitHub Actions CI/CD and a runnable demonstration environment.

g) Proposed Tasks:

- **TP1: Requirements, Architecture & Experimental Design (Weeks 1 to 3):** SRS, state machine diagrams, the architecture document, analysis of the inherited firmware message schema and event ID structure, database design, the experiment protocol and a proof-of-concept serial parser. Later task packages depend on the message schema frozen here.
- **TP2: Field Station & Offline Synchronization (Weeks 2 to 6):** serial integration with the attached node, the SQLite priority queue, MQTT publishing, reconnection detection and flush, health reporting, and unit and integration tests for queueing, ordering and recovery.
- **TP3: Core Backend & Reliability Services (Weeks 3 to 7):** authentication and RBAC, device and contract lifecycles, idempotent event ingestion, synchronization audit log, database migrations, and idempotency and concurrency tests.
- **TP4: Monitoring & Incident Management (Weeks 5 to 9):** telemetry ingestion, the organization-scoped live map and API, the SOS-to-incident pipeline, tiered alerts and the incident audit trail.
- **TP5: Business Operations (Weeks 6 to 10):** organization onboarding, rental contracts, reservation, handover and check-in, inspection, billing, payments, dashboards and reports.
- **TP6: Integration, Evaluation & Deployment (Weeks 9 to 13):** physical experiments for RQ1 and RQ2 over the connectivity-loss matrix, randomized SOS drills for RQ3 with automatic timestamp recording, cross-organization isolation and load tests, the evaluation report, end-to-end tests, deployment to AWS with Docker Compose and GitHub Actions, and final documentation.

Risk assessment:

- **Too few physical TrekLink devices** (Likelihood: Medium / Impact: High): Simulate additional Field Station load with MQTT scripts. Physical devices are used for the mesh-to-Field Station measurements, and simulation is not used to claim LoRa radio reliability.
- **Synchronization latency above the target** (Likelihood: Medium / Impact: Medium): Tune the MQTT QoS level, reduce the payload size and pace the queue flush.
- **RQ3 baseline hard to measure objectively** (Likelihood: High / Impact: Medium): Run randomized drills in which an independent observer triggers the SOS and records the time.

- **NestJS, MQTT and WebSocket integration underestimated** (Likelihood: Medium / Impact: High): Build an early proof of concept from the Field Station to the backend in TP1.
- **Firmware message schema changes** (Likelihood: Low / Impact: High): Freeze and document the schema in TP1.
- **Scope creep** (Likelihood: High / Impact: Medium): Freeze features after TP5 and move further features to the post-Capstone backlog.

3.3. Research Information

a. Research Problem / Research Question

Organizations that operate where cellular coverage is limited need field events, including SOS alerts, to reach their staff after an outage of minutes or longer. They also need each SOS to become a coordinated incident with an owner, a tracked acknowledgment and an audit trail.

Prior work covers parts of this need separately: LoRa mesh systems relay messages in the field, IoT platforms manage connected devices, and incident tools manage escalation, as section 3.3 e shows.

This Capstone proposes a Field Station with priority-aware offline queuing and idempotent event processing, and an SOS-to-incident pipeline with a defined lifecycle and tiered escalation. It evaluates both through three research questions:

- **RQ1:** How reliably can the TrekLink Field Station synchronize SOS, GPS and telemetry events from an intermittently connected LoRa mesh to the cloud backend, measured by message delivery rate, data-loss rate and duplicate rate?
- **RQ2:** How do connectivity-loss duration and reconnection pattern affect delivery rate, synchronization latency and priority-ordering compliance?
- **RQ3:** Compared with the uncoordinated device-level SOS workflow, to what extent does the SOS-to-incident pipeline improve response coordination and traceability, measured by MTTA, MTTR, traceability score and incident-completion rate in controlled drills?

RQ1 and RQ2 evaluate the Event Reliability & Offline Synchronization requirements. RQ3 evaluates the incident management requirements and the incident lifecycle.

Baseline for RQ3: a device broadcasts an SOS over the mesh, and a guide whose phone receives it calls or messages staff. This workflow has no structured acknowledgment, timestamp record or response log. Baseline MTTA is the time from the SOS trigger to the first acknowledgment, recorded by an independent observer. The baseline traceability score is 0%, because the workflow keeps no audit record. A fall in MTTA is not assumed in advance, and outcomes are reported as measured.

b. Research Objectives

- Design and implement a priority-aware, offline-resilient Field Station with idempotent event processing that connects the TrekLink LoRa mesh to a cloud backend.

- Evaluate synchronization reliability under a controlled matrix of connectivity-loss durations and reconnection patterns, with physical TrekLink hardware for the mesh-to-Field Station measurements.
- Design and implement an SOS-to-incident pipeline with a defined lifecycle, tiered notification and escalation, and an append-only audit trail.
- Evaluate the incident workflow by MTTA, MTTR, traceability score and incident-completion rate in randomized drills, against the baseline.
- Integrate both mechanisms into a multi-organization rental platform with asset management, rental contracts, live telemetry and billing.

c. Research Scope & Methodology

In scope: TrekLink ESP32/ESP32-S3 devices and the LoRa mesh with targeted firmware enhancements; the Field Station with its priority queue and idempotent synchronization; SOS, GPS and selected telemetry events; connectivity-loss experiments in a controlled lab; SOS drills; the organization, asset, contract, incident, telemetry and billing modules with multi-tenancy; a responsive web client; sandbox online payment.

Out of scope: rearchitecting the mesh stack, native mobile apps, live commercial payment, hardware radio certification, individual renters, and the organizations' own trip and rescue operations.

Methodology:

- **Literature Review:** Survey LoRa mesh systems [1], [2], store-and-forward and delay-tolerant protocols [5], IoT fleet management [12] and emergency incident workflows [3], [13], and state the gap between them.
- **Architecture & Design:** Freeze the message schema and event ID structure, then design the synchronization architecture, priority queue, idempotency store, incident lifecycle and experiment protocol.
- **Implementation:** Build the Field Station, backend modules and web client in TP1 to TP5.
- **Experiment 1, Synchronization for RQ1 and RQ2:** Use 3 to 5 physical TrekLink devices. Apply connectivity losses from 0 s as the control up to 30 min, with immediate and delayed reconnection, and at least 20 trials per condition. MQTT scripts may add backend load only. Measures: delivery rate, data-loss rate, duplicate rate, synchronization latency as mean, median and P95, queue-recovery time and priority-ordering compliance.
- **Experiment 2, SOS-to-Incident Workflow for RQ3:** Run randomized drills with at least two organizations, each with a device, a Field Station and on-duty members. An independent observer triggers the SOS at a time the participants do not know. The backend records the TrekLink timestamps, and the observer times the baseline with a stopwatch. Run 15 to 20 drills per condition over LAN and a 4G hotspot. Measures: MTTA, MTTR, traceability score, missed-alert rate, duplicate-alert rate, incident-completion rate and cross-organization leakage.

- **Integration & System Testing:** End-to-end tests of the functional requirements, idempotency and concurrency tests, RBAC checks, an audit-log completeness check, a cross-organization isolation suite and a load test with simulated devices and Field Stations.

d. Expected Scientific Contribution

- **A priority-aware, offline-resilient gateway pattern** that connects intermittently connected LoRa mesh devices to a cloud platform, with a defined event ID scheme, local persistence and idempotent processing.
- **An empirical benchmark** of synchronization reliability and latency under controlled connectivity loss on low-cost ESP32 hardware.
- **An SOS-to-incident workflow design** with a specified lifecycle, tiered escalation and an audit trail, traceable to measurable outcomes.
- **A comparative evaluation** of uncoordinated and structured incident response by MTTA, MTTR and traceability.
- **A case study** of integrating an open-source LoRa mesh prototype into a multi-organization device rental platform for trekking agencies.

e. Related Works / Literature Review (Preliminary)

- **LoRa mesh communication** such as Meshtastic and RAK WisBlock Mesh [1], [2], [6]: multi-hop messaging, GPS, local SOS broadcast and fall detection without infrastructure. Gap: no organization-side rental and fleet management, no store-and-forward to the cloud and no incident pipeline.
- **IoT fleet management platforms** such as AWS IoT Core, Azure IoT Hub and Balena [12]: device registration, telemetry ingestion and remote monitoring. Gap: no LoRa mesh integration, no store-and-forward for intermittent connectivity and no rental domain.
- **Delay-tolerant and store-and-forward systems** such as the DTN Bundle Protocol, MQTT-SN and LoRaWAN [5], [10], [11]: message persistence and forwarding under intermittent connectivity. Gap: protocol-level designs with no business workflow or incident management.
- **Incident management systems** such as Jira Service Management, PagerDuty and Opsgenie [13]: ticket lifecycle, escalation, notification and audit trail. Gap: built for IT operations, with no field-event ingestion and no outdoor context.

Gap statement: TrekLink combines off-grid LoRa mesh communication, store-and-forward synchronization, device rental lifecycle management, live field monitoring and structured SOS-to-incident response in one domain-specific platform.

Key References:

- [1] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, “Multi-hop and mesh for LoRa networks: Recent advancements, issues, and recommended applications,” ACM Comput. Surv., vol. 56, no. 6, Article 136, pp. 1–43, Jan. 2024. DOI: 10.1145/3638241

- [2] Muladi, H. Wijaya, S. D. Prasetyo, S. A. Hamzah, and A. K. Mahamad, “LoRa mesh-based IoT GPS tracking system for mountain climbers,” Int. J. Safety Security Eng., vol. 14, no. 6, pp. 1751–1761, Dec. 2024. DOI: 10.18280/ijsse.140610
- [3] H. Zhang, R. Zhang, and J. Sun, “Developing real-time IoT-based public safety alert and emergency response systems,” Sci. Rep., vol. 15, Article 29056, Aug. 2025. DOI: 10.1038/s41598-025-13465-7
- [4] OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- [5] V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall, and H. Weiss, “Delay-tolerant networking architecture,” IETF RFC 4838, Apr. 2007. https://www.rfc-editor.org/rfc/rfc4838
- [6] Meshtastic Project, “Official Documentation,” 2024. [Online]. Available: https://meshtastic.org/docs/
- [7] W. Vogels, “Eventually consistent,” Commun. ACM, vol. 52, no. 1, pp. 40–44, Jan. 2009. DOI: 10.1145/1435417.1435432
- [8] D. Harel, “Statecharts: A visual formalism for complex systems,” Sci. Comput. Program., vol. 8, no. 3, pp. 231–274, Jun. 1987. DOI: 10.1016/0167-6423(87)90035-9
- [9] P. Helland, “Idempotence is not a medical condition,” Commun. ACM, vol. 55, no. 5, pp. 56–65, May 2012. DOI: 10.1145/2160718.2160734
- [10] A. Stanford-Clark and H. L. Truong, “MQTT For Sensor Networks (MQTT-SN) Protocol Specification, Version 1.2,” IBM, Nov. 2013. [Online]. Available: https://www.oasis-open.org/committees/download.php/66091/MQTT-SN_spec_v1.2.pdf
- [11] LoRa Alliance, “TS001-1.0.4 LoRaWAN L2 1.0.4 Specification.” [Online]. Available: https://resources.lora-alliance.org/technical-specifications/ts001-1-0-4-lorawan-l2-1-0-4-specification
- [12] Amazon Web Services, “What is AWS IoT?,” AWS IoT Core Developer Guide. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html
- [13] PagerDuty, “Escalation Policy Basics,” PagerDuty Knowledge Base. [Online]. Available: https://support.pagerduty.com/main/docs/escalation-policies
- [14] Garmin Ltd., “inReach Mini 2 satellite communicator,” product page. [Online]. Available: https://www.garmin.com/en-US/p/837461/pn/010-02602-00

4. Other comments (propose all relative things if have):

The TrekLink LoRa mesh firmware and hardware for ESP32/ESP32-S3 come from this team’s Summer 2026 project and are the technical foundation of this Capstone. Targeted firmware enhancements made during the Capstone are in scope and credited to it. The reports document the hardware design work.

Supervisor (If have) ***(Sign and full name)***	**HCM, date ____/____/2026** **On behalf of Registers** ***(Sign and full name)***